

AI FAQ ASSISTANT API

Phase- 3 : PROJECT DESIGN

Project Title: AI FAQ Assistant API

3. PROJECT DESIGN

Project design converts the requirements of the AI FAQ Assistant API into a technical structure that can be implemented and tested. The design explains the components of the system, the movement of information between components, the API interface, data storage, input and output formats, security controls, and error-handling methods. A clear design reduces development confusion and provides a common reference for all project members.

The AI FAQ Assistant API is designed as a modular application. A client sends a question through an HTTP request. The API validates the request, processes the question, searches the FAQ knowledge source or uses an AI component, and returns a structured JSON response. The separation of these activities allows individual modules to be changed without redesigning the entire system.

3.1 Design Objectives

- Provide a simple API for submitting frequently asked questions.
- Return clear and structured answers in JSON format.
- Separate request handling, question processing, data access, and response generation.
- Support future integration with web, mobile, or desktop applications.
- Provide secure input validation and controlled error handling.
- Make the system easy to test, maintain, and extend.

3.2 System Architecture

The proposed architecture contains client, API, processing, knowledge, data, and security responsibilities. The client can be a website, mobile application, Postman, or another software system. The API layer receives HTTP requests and sends responses. The processing layer validates and normalizes the question. The knowledge layer searches FAQ records and, where required, connects to an AI service. The response layer creates the final JSON response.

Layer	Responsibility
-------	----------------

Client Layer	Sends questions and receives responses.
API Layer	Provides HTTP endpoints and controls request/response communication.
Processing Layer	Validates, cleans, and prepares questions.
Knowledge/AI Layer	Finds relevant FAQ information or generates an answer.
Data Layer	Stores questions, answers, categories, and metadata.
Security Layer	Protects restricted operations and validates access.

3.3 Component Design

The API server is the entry point of the application. It receives a request and routes it to the correct operation. The validation component checks the request before processing. The FAQ component retrieves information from the knowledge source. The AI component may interpret natural-language variations and assist with answer generation. The response component produces a consistent output. Logging and error handling provide information required for maintenance.

- API Server – manages HTTP requests and endpoint routing.
- Validation Module – checks required fields, data types, and input limits.
- FAQ Retrieval Module – searches FAQ records for relevant information.
- AI Processing Module – supports natural-language understanding and answer generation.
- Database Module – stores and retrieves structured FAQ data.
- Response Module – creates consistent success and error responses.
- Logging Module – records technical events for debugging and monitoring.

3.4 Data Flow Design

The main data flow begins when a client sends a question. The API receives the request and validates the JSON structure. After validation, the question is normalized and passed to the retrieval or AI processing module. The module identifies relevant knowledge and prepares an answer. The response is formatted as JSON and returned to the client.

If the request is invalid, the processing flow stops and an error response is generated. If the question cannot be answered confidently, the system can use a controlled fallback message. This prevents the API from returning an unsupported response when the required information is unavailable.

3.5 Use Case Design

Use Case	Actor	Description
Ask Question	API Client	Submits a question and receives an answer.
Validate Request	System	Checks the request structure and values.
Search FAQ	System	Finds relevant FAQ records.
Generate Answer	System/AI	Creates or selects the appropriate answer.
Manage FAQ	Administrator	Adds, updates, or removes FAQ information.
Check Service	API Client	Checks whether the API is available.

3.6 Database Design

FAQ data should be stored separately from application logic. A database such as SQLite or PostgreSQL can be used for structured storage. A simple prototype may use JSON data, while a

larger deployment can use a relational database. Each FAQ record should have a unique identifier and information needed for retrieval and maintenance.

Field	Type	Purpose
faq_id	Integer/String	Unique FAQ identifier.
question	Text	Frequently asked question.
answer	Text	Approved answer or knowledge content.
category	Text	Groups related questions.
keywords	Text	Supports search and matching.
created_at	Date/Time	Record creation time.
updated_at	Date/Time	Last modification time.

3.7 API Design

The API uses HTTP endpoints and JSON as the primary data exchange format. A POST operation is appropriate for submitting a question because the question is included in the request body. GET operations can be used for health checks or retrieving permitted information. Administrative operations should be protected.

Endpoint	Method	Purpose
/ask	POST	Processes a user question.
/health	GET	Checks whether the service is running.
/faqs	GET	Retrieves FAQ information when permitted.
/faqs	POST	Adds FAQ information for

		authorized users.
--	--	-------------------

Example request: {"question":"How can I reset my password?"}. A successful response may contain fields such as success, answer, category, and source. The exact response structure can be adjusted during implementation.

3.8 Input Design

The main input is a natural-language question. Input validation is necessary because invalid requests can cause unnecessary processing or unexpected errors. The API should check for a missing question, empty text, invalid JSON, unsupported data types, and excessive input length.

- The question field must be present.
- The question must contain meaningful text.
- Input length should be limited.
- JSON syntax and content type must be validated.
- Unexpected input should be handled safely.

3.9 Output Design

The output should be predictable and easy for both people and programs to use. JSON is selected because it is lightweight and widely supported. A success response may include the answer and category. An error response should include a status, error code, and readable message.

Example: {"success":true,"answer":"You can reset your password from account settings.","category":"Account Support"}.

3.10 Module Design

Module	Main Function
API Module	Defines endpoints and handles HTTP communication.

Validation Module	Validates request data.
FAQ Module	Reads and searches FAQ content.
AI Module	Processes natural language when required.
Database Module	Stores and retrieves data.
Response Module	Creates JSON responses.
Security Module	Protects restricted operations.
Logging Module	Records important technical events.

3.11 Security Design

Security is included at the API boundary and inside the processing flow. Authentication can be applied to administrative endpoints. Secret API keys should be stored in environment variables instead of source code. HTTPS should be used during production deployment to protect information while it is transmitted.

The application should validate user input and avoid exposing internal stack traces. Logs should not contain passwords or secret keys. Rate limiting can be added to reduce excessive requests. Access control should ensure that only authorized users can change FAQ records.

3.12 Error Handling Design

The API should return controlled errors instead of exposing internal implementation details. Common errors include invalid input, unauthorized access, missing resources, external AI-service failures, database problems, and unexpected server errors.

Condition	Status	Meaning
Invalid input	400	Request data is invalid.
Unauthorized	401	Authentication is missing or invalid.

Forbidden	403	Operation is not permitted.
Not Found	404	Requested resource does not exist.
Server Error	500	Unexpected internal problem.

3.13 Technology Design

Python is suitable for the project because it provides libraries for web APIs, data processing, natural-language processing, testing, and database connectivity. FastAPI or Flask can be used as the API framework. Visual Studio Code can be used for development, while Git can provide version control. Postman or Swagger/OpenAPI can be used to test and document endpoints.

The project should use a virtual environment to isolate dependencies. A requirements file should list the required packages. Configuration values should be supplied through environment variables. The final deployment environment can be selected according to project requirements.

3.14 Deployment Design

The deployment design consists of the application server, configuration, database, and optional AI service. Before deployment, dependencies must be installed and configuration values must be set securely. A health endpoint can be used to verify that the service has started correctly. After deployment, sample requests should be tested to confirm the complete request-to-response flow.

3.15 Design Advantages

- Modular structure makes development and maintenance easier.
- Standard JSON responses simplify integration with other applications.
- Separate data storage allows FAQ content to be updated independently.
- Validation and error handling improve reliability.
- Security controls protect restricted operations.

- The architecture supports future AI and multilingual enhancements.

3.16 Summary

The project design establishes the technical foundation for the AI FAQ Assistant API. It defines the architecture, components, data flow, use cases, database structure, API interface, input and output formats, modules, security, errors, technology, and deployment approach. This design can be used as the reference during implementation and testing.